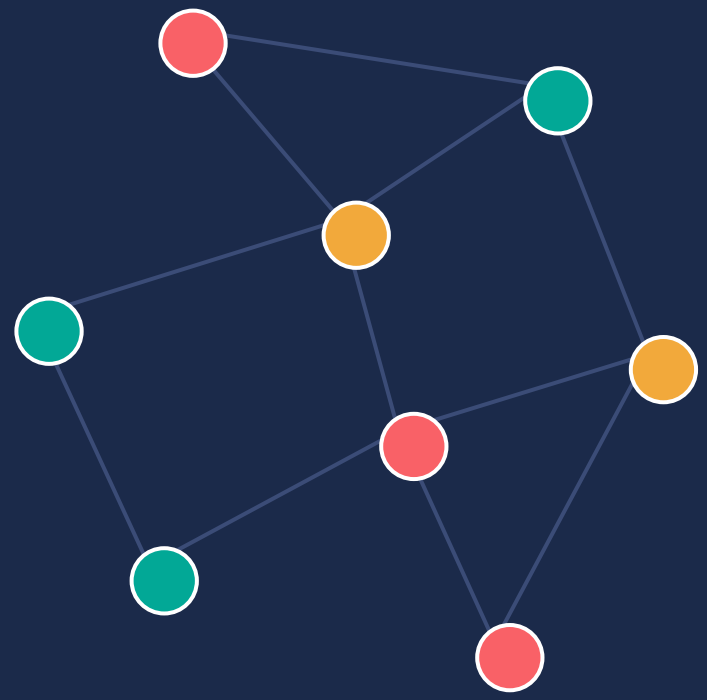
REVIEW 2 · IMPLEMENTATION, TESTING & DEPLOYMENT

Graph-Coloring Based Register Allocator

The plan from Review 1, built: a Chaitin–Briggs allocator that parses, colours, spills, and passes 87 tests — live on the web.



1. Sandeep More 24BCE2883
2. Shantanu Saurav 24BAI0316
3. Kunwar Vishwameet Singh 24BAI0235
4. Pratyush Chakraborty 24BAI0104

RECAP

From Plan to Pipeline

1 **Parse IR and build the CFG** ✓

src/frontend — tokenizer, recursive-descent parser, basic-block + successor/predecessor graph, loop-depth

2 **Perform liveness (data-flow) analysis** ✓

src/analysis/liveness.py — backward fixed-point dataflow, IN/OUT sets per block

3 **Construct interference graphs** ✓

src/analysis/interference.py — adjacency built from live ranges, plus move-pairs for coalescing

4 **Assign registers using graph coloring** ✓

src/colouring + src/spilling — Briggs coalescing, simplify/select, cost-based spill, rewrite/retry loop

SCOPE HELD

Intra-procedural allocation over custom three-address code — exactly the scope set in Review 1. No inter-procedural allocation, no target ISA.

87 / 87

unit tests passing, one suite per module

LIVE

Deployed as a working web app — not just a CLI.

ARCHITECTURE

The Pipeline, End to End

1 **Parse**
TAC → instructions

→

2 **CFG Build**
blocks, loop depth

→

3 **Liveness**
IN / OUT, fixed point

→

4 **Interference**
graph + move pairs

→

5 **Coalesce**
Briggs conservative test

→

6 **Colour**
simplify → select

↺

Not enough colours? Spill & retry.

A node with no free colour is spilled to memory (min cost/degree); stores + fresh-name loads are inserted, and the whole pipeline — CFG → liveness → interference → colour — runs again. Capped at 10 rounds; every benchmark converges in 1–4.

7 **Output**

Allocated program with physical registers assigned, or spill loads/stores rewritten in — both are valid, correct output.

src/ir.py (Instr, BasicBlock, Function) and src/analysis/interference.py (InterferenceGraph) are the two contracts every module crosses through — unchanged since Review 1.

GROUNDWORK

The IR, and the CLI Around It

ACCEPTED IR FORMS

`dst = const`	`t1 = 5`	load a constant
`dst = src`	`t2 = t1`	copy (coalescing candidate)
`dst = src1 OP src2`	`t3 = t1 + t2`	binary operation
`if src REL src goto L`	`if t5 < 100 goto L1`	conditional jump
`label / goto / ret / func`	`label L1`	control-flow & boundaries

Operators: + − × ÷ % · Relations: < ≤ > ≥ == ≠ · vregs: t_1, t_2, …

```
$ python -m src.cli
```

- **`--k N`** — physical registers (default 8)
- **`--dump-cfg`** — print parsed instructions + CFG
- **`--dump-live`** — print live_in / live_out per block
- **`--dump-graph FILE`** — write interference graph as Graphviz DOT
- **`--no-coalesce`** — disable coalescing — O3 baseline

87 pytest cases + a GitHub Actions workflow run the same CLI path on every push — the tool a reviewer runs by hand is exactly the tool CI checks.

WALKTHROUGH

A Worked Example, End to End

benchmarks/briggs_example.tac · 8 virtual registers · K = 3

```
t1 = 1
t2 = 2
t3 = t1 + t2
t4 = t1 + t3
t5 = t2 + t3
t6 = t4 + t5
t7 = t4 + t6
t8 = t5 + t7
ret t8
```

RESULT: EVERY REGISTER COLOURED, ZERO SPILLS

t1 → R2	t2 → R1	t3 → R0	t4 → R2
t5 → R1	t6 → R0	t7 → R0	t8 → R0

A bug we caught before it shipped

Our hand-worked interference graph (docs/briggs_example_solution.md) was missing the t2–t4 edge — t2 stays live right through t4's definition, so they interfere even though neither instruction names the other. A dedicated regression test now pins the corrected 9-edge graph exactly.

UNDER PRESSURE

Spilling in Action

Same program, tighter budget: benchmarks/briggs_example.tac at K = 2

4	14	10
iterations to converge	spill loads/stores inserted	max retry cap (safety net)

cost / degree

cost = Σ 10^(loop depth) over every def/use — a hot-loop value is protected; a value used once is cheap to spill.

BEFORE

```
t3 = t1 + t2
t4 = t1 + t3
t5 = t2 + t3
t6 = t4 + t5
t7 = t4 + t6
t8 = t5 + t7
```

AFTER — REWRITTEN WITH FRESH-NAME RELOADS

```
spill[0] = t3
t9 = spill[0]
t4 = t1 + t9
spill[1] = t4
t10 = spill[0]
t5 = t14 + t10
… (fresh vreg per reload, one live for 1-2 instrs)
```

RIGOR

Three Bugs We Found — and What They Taught Us

!

A missing interference edge

Our own hand-worked graph for briggs_example.tac omitted t2–t4. Fixed in the doc, and locked down by a dedicated regression test on the corrected 9-edge set.

!

A cost heuristic that never converges

Feeding spill_cost into the optimistic push looks safe — until K=2 on briggs_example.tac: it protects the one expensive node causing all the pressure and thrashes on cheap temporaries forever (40 rounds, no convergence). Plain highest-degree converges in 4.

!

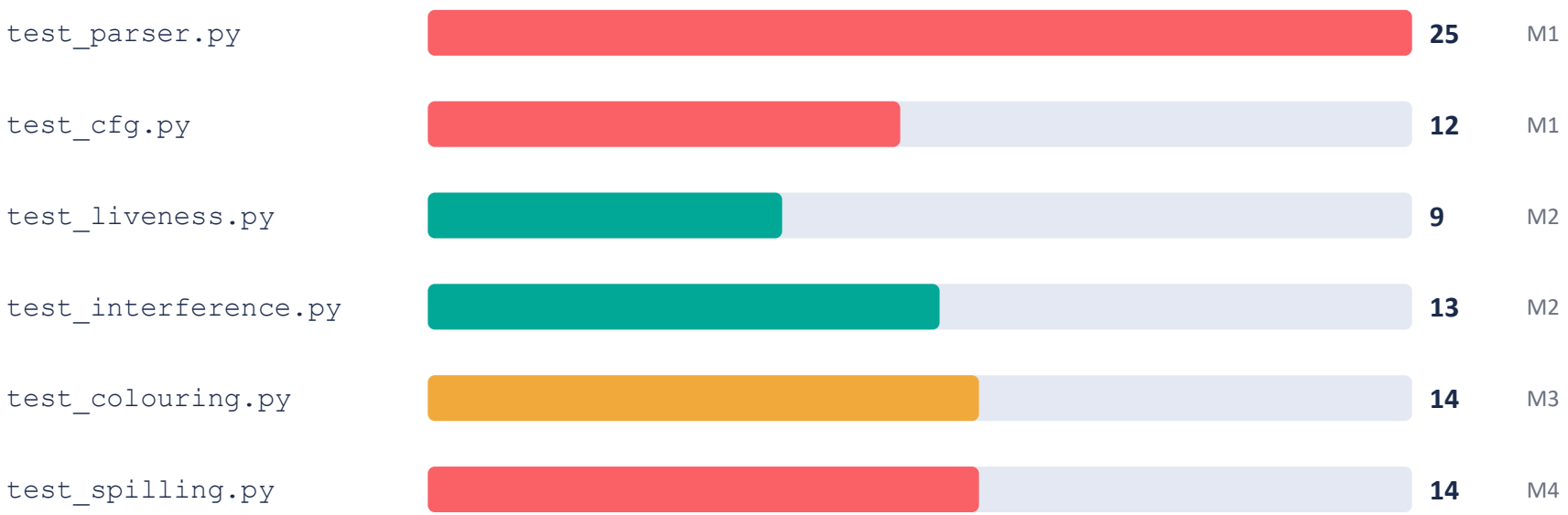
The deployed API skipped a step

The web API built the interference graph straight from the parser, before the CFG or liveness ever ran — correct only for straight-line code. Every benchmark with a branch or loop would have been misanalysed. Fixed by running build_cfg + analyse first, matching the CLI.

Review 1 flagged “infinite spill loops” and “incorrect interference graphs” as top risks. Both showed up during implementation — exactly as anticipated — and both are now caught by tests, not by luck.

VERIFICATION

87 Tests, One Suite Per Module



87 / 87

PASSING

GitHub Actions runs pytest on every push and pull request.

Coverage includes: fixed-point liveness with a diamond join, the hand-verified interference edge set, simplify/select at K=1..4 (incl. the optimistic-push case), a safe Briggs merge and a correctly-refused interfering one, spill cost's loop-depth weighting, the degree-0 guard, fresh-name reload generation, and full-pipeline convergence on every benchmark.

RESULTS

Every Benchmark, Every K

6 benchmark programs × K ∈ {1, 2, 3, 4, 8} — 30 runs, exercised by both the CLI and the test suite

Benchmark	Shape	K=2	K=3	K=4	K=8
`simple.tac`	5 vregs, straight-line	0	0	0	0
`diamond.tac`	if/else join	0	0	0	0
`loop.tac`	single loop	0	0	0	0
`nested_loop.tac`	loop inside loop	9 (3 it.)	0	0	0
`briggs_example.tac`	Briggs (1994) fixture	14 (4 it.)	0	0	0
`high_pressure.tac`	8 values live at once	14 (3 it.)	12 (3 it.)	10 (3 it.)	0

K = 1 fails on all six benchmarks — architecturally, not a bug: every binary op and if..goto needs two operands live at once, so one register is never enough. Every benchmark succeeds at K ≥ 2, and the retry loop never takes more than 4 rounds.

OPTIMIZATION

Coalescing: Verified Correct, Not Yet Proven Profitable

HOW IT'S TESTED

- Briggs' conservative test: merge only if the combined node has < K neighbours of significant degree — provably never creates an uncolourable node.
- test_briggs_allows_a_safe_merge — a safe copy pair coalesces.
- test_briggs_refuses_to_merge_interfering_nodes — an unsafe pair is correctly refused.
- test_coalesce_transfers_the_merged_nodes_edges — the merged node's neighbours are preserved exactly.

WHAT WE MEASURED (O3 TARGET: −20% SPILLS)

Benchmark	With	Without
simple / diamond / loop / nested_loop / briggs_example	0	0
high_pressure.tac (K=4)	10	10

Coalescing removes copies correctly, but none of our six benchmarks contain a copy pair whose removal relieves register pressure — so the spill count doesn't move. This is a gap in our benchmark suite, not in the algorithm.

Next step: add a copy-heavy benchmark to demonstrate O3 quantitatively.

DEPLOYMENT

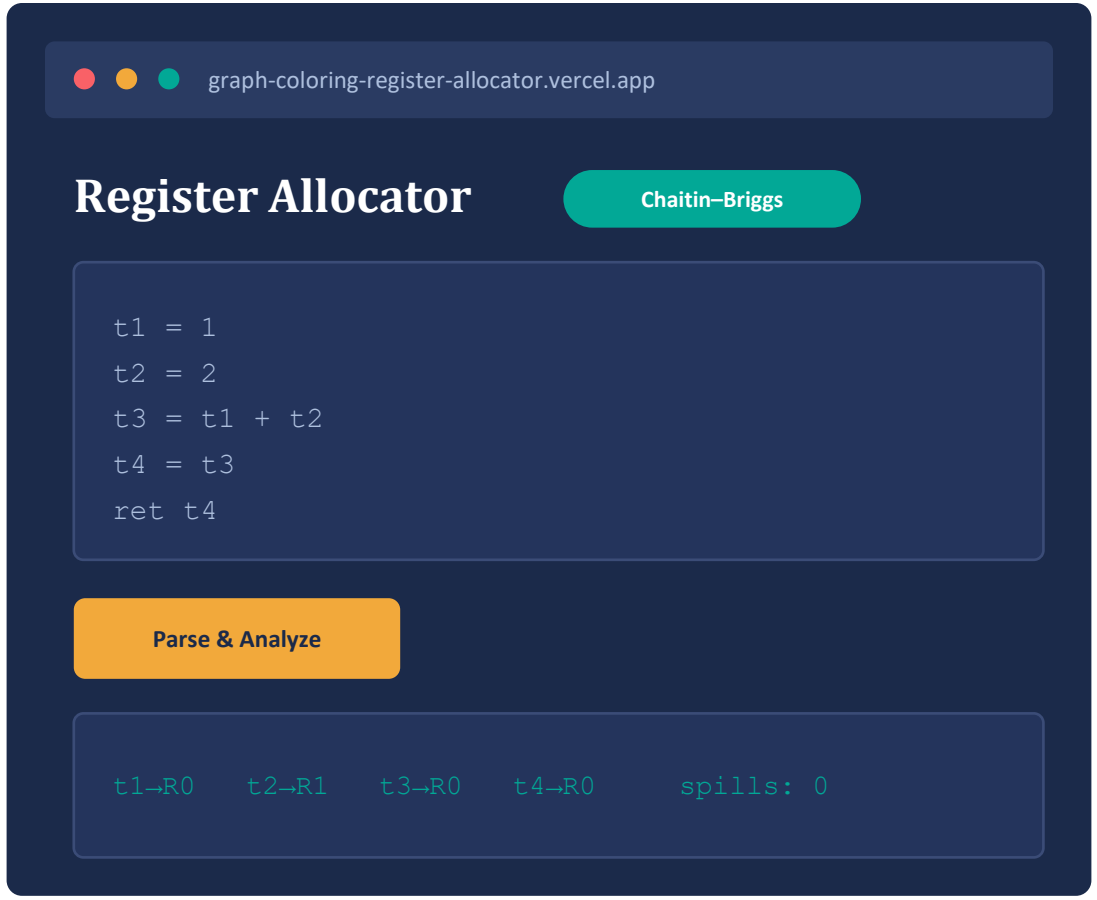
From CLI to a Live Web App

VERCEL · FASTAPI + STATIC FRONTEND

- **`api/index.py`**
  FastAPI endpoint running the real pipeline
- **`public/index.html + script.js`**
  paste TAC → pick K → Parse & Analyze
- **`vercel.json`**
  serverless routing config

A live-only bug we found (and fixed)

The API built the interference graph directly from the parser, before the CFG/liveness ever ran — silently wrong on anything with a branch or loop. Fixed by routing through build_cfg + analyse, exactly like the CLI.



HOW WE WORK

Process, Ownership & CI Gate

M1 ✓

frontend/

Parse, CFG, loop depth

M2 ✓

analysis/

Liveness, interference graph

M3 ✓

colouring/

Coalescing, simplify/select

M4 ✓

spilling/

Cost model, rewrite, retry loop

WORKFLOW

1. One branch per module: m1-frontend, m2-analysis, m3-colouring, m4-spilling.
2. Every merge goes through a pull request reviewed by another member.
3. CI runs pytest on every push and PR — the suite must pass before merging.
4. M4 owns main and the regression suite end to end.

.github/workflows/ci.yml

```
on: push, pull_request
runs-on: ubuntu-latest
python-version: "3.11"
pip install -r requirements.txt
pytest
```

✓ Passing on every push since implementation began.

OUTLOOK

Objectives Met, and What's Next

OBJECTIVES FROM REVIEW 1 — STATUS

- ✓ Parser + CFG + loop depth — done, 37 tests
- ✓ Liveness analysis (fixed-point) — done, 9 tests
- ✓ Interference graph construction — done, 13 tests, hand-verified edge set
- ✓ Briggs coalescing, simplify/select colouring — done, 14 tests
- ✓ Cost-based spilling + rewrite/retry loop — done, 14 tests, converges in ≤ 4 rounds
- ✓ Deployed as a live, working web app on Vercel

FUTURE EXTENSIONS

1. A copy-heavy benchmark to quantitatively demonstrate coalescing's spill reduction (O3).
2. Register coalescing extended with iterated (George & Appel) coalescing between rounds.
3. Inter-procedural allocation across function boundaries.
4. Target-specific ISA constraints (calling convention, callee-saved sets).

REFERENCES

1. Chaitin, G. J. (1982). Register allocation & spilling via graph coloring. SIGPLAN.
2. Briggs, P., Cooper, K. D., Torczon, L. (1994). Improvements to graph coloring register allocation. ACM TOPLAS 16(3).
3. George, L., Appel, A. W. (1996). Iterated register coalescing. ACM TOPLAS 18(3).
4. Aho, A. V. et al. (2006). Compilers: Principles, Techniques, and Tools, 2nd ed.

---

github.com/ShantanuSaurav/Graph-Coloring-Register-Allocator

graph-coloring-register-allocator.vercel.app

Thank You

Any Questions?